Below is your **Apache Web Server configuration converted from RHEL/CentOS to Ubuntu-based systems** (Ubuntu 20.04, 22.04, and 24.04).

The main differences are:

- RHEL: `httpd` → Ubuntu: `apache2`
- RHEL: `/etc/httpd/` → Ubuntu: `/etc/apache2/`
- RHEL: `/var/www/html` → Ubuntu: `/var/www/html`
- RHEL: `yum` → Ubuntu: `apt`
- RHEL: `systemctl start httpd` → Ubuntu: `systemctl start apache2`

1. Install Apache Web Server on Ubuntu

Update the package repository:

bash

```
None
sudo apt update
```

Install Apache:

bash

```
None
sudo apt install apache2 -y
```

Check Apache status:

bash

```
None
sudo systemctl status apache2
```

If Apache is stopped, start it:

bash

```
None
sudo systemctl start apache2
```

Enable Apache on system boot:

bash

```
None
sudo systemctl enable apache2
```

Check Apache version:

bash

```
None
apache2 -v
```

2. Generate SSL Certificate Using Certbot

Install Certbot:

bash

```
None
sudo apt install certbot python3-certbot-apache -y
```

Stop Apache:

bash

```
None
sudo systemctl stop apache2
```

Generate an SSL certificate using standalone mode:

bash

```
None
sudo certbot certonly --standalone -d devlinux.xyz
```

After successful certificate generation, your certificates will be available at:

bash

```
None
/etc/letsencrypt/live/devlinux.xyz/fullchain.pem
/etc/letsencrypt/live/devlinux.xyz/privkey.pem
```

- `fullchain.pem` → SSL certificate (public certificate chain).
- `privkey.pem` → SSL private key.

Note: Your domain must point to your server's public IP, and ports 80 and 443 must be accessible.

3. Configure Apache for a Single Website with SSL

Enable the SSL module:

bash

```
None
sudo a2enmod ssl
```

Open the default Apache configuration:

bash

```
None
sudo vim /etc/apache2/sites-available/000-default.conf
```

Alternatively, create a separate SSL configuration:

bash

```
None
sudo vim /etc/apache2/sites-available/devlinux.xyz.conf
```

Add the following configuration:

apache

```
None
<VirtualHost *:443>

    ServerName devlinux.xyz
    ServerAlias www.devlinux.xyz

    DocumentRoot /var/www/html

    SSLEngine on
```

```
    SSLCertificateFile
/etc/letsencrypt/live/devlinux.xyz/fullchain.pem
    SSLCertificateKeyFile
/etc/letsencrypt/live/devlinux.xyz/privkey.pem

    ErrorDocument 404 /error_404.html

    DirectoryIndex index.html

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined

</VirtualHost>
```

Enable the SSL site:

bash

```
None
sudo a2ensite devlinux.xyz.conf
```

Enable SSL module (if not already enabled):

bash

```
None
sudo a2enmod ssl
```

Check configuration:

bash

```
None
sudo apache2ctl configtest
```

Restart Apache:

bash

```
None
sudo systemctl restart apache2
```

4. Configure Apache for Hosting Multiple Websites

In Ubuntu, Apache uses the following directory structure:

Purpose	Ubuntu path
Main configuration	`/etc/apache2/apache2.conf`
Available websites	`/etc/apache2/sites-available/`
Enabled websites	`/etc/apache2/sites-enabled/`
Available modules	`/etc/apache2/mods-available/`
Enabled modules	`/etc/apache2/mods-enabled/`
Document root	`/var/www/`
Log files	`/var/log/apache2/`

Unlike RHEL, Ubuntu uses `a2ensite` and `a2dissite` to enable and disable websites.

Website 1: devlinux.xyz

Create a website directory:

bash

```
None
sudo mkdir -p /var/www/devlinux.xyz
```

Create an index file:

bash

```
None
sudo vim /var/www/devlinux.xyz/index.html
```

Create the virtual host configuration:

bash

```
None
sudo vim /etc/apache2/sites-available/devlinux.xyz.conf
```

Configuration:

apache

```
None
<VirtualHost *:443>

    ServerName devlinux.xyz
    ServerAlias www.devlinux.xyz

    DocumentRoot /var/www/devlinux.xyz

    SSLEngine on
    SSLCertificateFile
/etc/letsencrypt/live/devlinux.xyz/fullchain.pem
    SSLCertificateKeyFile
/etc/letsencrypt/live/devlinux.xyz/privkey.pem

    ErrorDocument 404 /error_404.html

    DirectoryIndex index.html

    ErrorLog ${APACHE_LOG_DIR}/devlinux_error.log
    CustomLog ${APACHE_LOG_DIR}/devlinux_access.log combined

</VirtualHost>
```

Enable the website:

bash

```
None
sudo a2ensite devlinux.xyz.conf
```

Website 2: website2.com

Create a website directory:

bash

```
None
sudo mkdir -p /var/www/website2
```

Create configuration:

bash

```
None
sudo vim /etc/apache2/sites-available/website2.conf
```

Add:

apache

```
None
<VirtualHost *:80>

    ServerName website2.com
    ServerAlias www.website2.com

    DocumentRoot /var/www/website2/

    ErrorLog ${APACHE_LOG_DIR}/website2_error.log
    CustomLog ${APACHE_LOG_DIR}/website2_access.log combined

</VirtualHost>
```

Enable the website:

bash

```
None
sudo a2ensite website2.conf
```

5. Configure Load Balancer for Website 2

First, enable the required Apache modules.

bash

```
None
sudo a2enmod proxy
```

```
sudo a2enmod proxy_http
sudo a2enmod proxy_balancer
sudo a2enmod lbmethod_byrequests
```

Open the website configuration:

bash

```
None
sudo vim /etc/apache2/sites-available/website2.conf
```

Add the following configuration:

apache

```
None
<VirtualHost *:80>

    ServerName website2.com
    ServerAlias www.website2.com

    DocumentRoot /var/www/website2/

    <Proxy "balancer://mycluster">

        BalancerMember http://13.234.21.220:80
        BalancerMember http://13.235.67.39:80

        ProxySet lbmethod=byrequests

    </Proxy>

    ProxyPass "/" "balancer://mycluster/"
    ProxyPassReverse "/" "balancer://mycluster/"

    ErrorLog ${APACHE_LOG_DIR}/website2_error.log
    CustomLog ${APACHE_LOG_DIR}/website2_access.log combined

</VirtualHost>
```

Check configuration:

bash

```
None
sudo apache2ctl configtest
```

Restart Apache:

bash

```
None
sudo systemctl restart apache2
```

How it works:

- Client sends a request to `website2.com`.
- Apache receives the request.
- Apache forwards the request to one of the backend servers.
- `lbmethod=byrequests` distributes requests based on the number of requests assigned to each backend.

6. Configure Another Website (site1.com)

Create the directory:

bash

```
None
sudo mkdir -p /var/www/site1/public_html
```

Create the configuration:

bash

```
None
sudo vim /etc/apache2/sites-available/site1.conf
```

Add:

apache

```
None
<VirtualHost *:80>

    ServerName site1.com
```

```
    ServerAlias www.site1.com

    DocumentRoot /var/www/site1/public_html

    ErrorLog ${APACHE_LOG_DIR}/site1_error.log
    CustomLog ${APACHE_LOG_DIR}/site1_access.log combined

</VirtualHost>
```

Enable the website:

bash

```
None
sudo a2ensite site1.conf
```

7. Enable and Disable Websites in Ubuntu

Enable a website:

bash

```
None
sudo a2ensite devlinux.xyz.conf
```

Disable a website:

bash

```
None
sudo a2dissite devlinux.xyz.conf
```

List enabled websites:

bash

```
None
ls /etc/apache2/sites-enabled/
```

List available websites:

bash

```
None
ls /etc/apache2/sites-available/
```

8. Restart and Verify Apache

Check Apache configuration:

bash

```
None
sudo apache2ctl configtest
```

Expected output:

text

```
None
Syntax OK
```

Restart Apache:

bash

```
None
sudo systemctl restart apache2
```

Check status:

bash

```
None
sudo systemctl status apache2
```

Reload Apache without a full restart:

bash

```
None
sudo systemctl reload apache2
```

Check Apache logs:

bash

```
None
sudo tail -f /var/log/apache2/error.log
```

9. Important Apache Commands: RHEL vs Ubuntu

Task	RHEL/CentOS	Ubuntu
Install Apache	`yum install httpd -y`	`apt install apache2 -y`
Start Apache	`systemctl start httpd`	`systemctl start apache2`
Stop Apache	`systemctl stop httpd`	`systemctl stop apache2`
Restart Apache	`systemctl restart httpd`	`systemctl restart apache2`
Check status	`systemctl status httpd`	`systemctl status apache2`
Enable on boot	`systemctl enable httpd`	`systemctl enable apache2`
Main config	`/etc/httpd/conf/httpd.conf`	`/etc/apache2/apache2.conf`
Virtual hosts	`/etc/httpd/conf.d/`	`/etc/apache2/sites-available/`
Document root	`/var/www/html`	`/var/www/html`
Logs	`/var/log/httpd/`	`/var/log/apache2/`
Enable SSL	Configuration/module	`a2enmod ssl`
Enable website	Manual configuration	`a2ensite`
Disable website	Manual configuration	`a2dissite`
Install Certbot	`yum install certbot`	`apt install certbot`

10. Important: Configure Automatic SSL Renewal

Let's Encrypt certificates expire after 90 days.

Check certificate renewal:

bash

```
None
sudo certbot renew --dry-run
```

Check Certbot timer:

bash

```
None
sudo systemctl status certbot.timer
```

Enable automatic renewal:

bash

```
None
sudo systemctl enable --now certbot.timer
```

Important: Since you generated your certificate using standalone mode, Apache must be stopped during standalone renewal. Configure a pre-hook and post-hook or switch to the Apache plugin for automatic renewal.

For Apache plugin renewal, you can use:

bash

```
None
sudo certbot --apache -d devlinux.xyz -d www.devlinux.xyz
```

---

11. Final Ubuntu Apache Directory Structure

After completing the configuration, your directory structure will look like this:

text

```
None
/etc/apache2/
|
├── apache2.conf
|
├── sites-available/
```

```
|   ├── devlinux.xyz.conf
|   ├── website2.conf
|   └── site1.conf
|
├── sites-enabled/
|   ├── devlinux.xyz.conf
|   ├── website2.conf
|   └── site1.conf
|
├── mods-available/
|
├── mods-enabled/
|
└── ports.conf


/var/www/
|
├── devlinux.xyz/
|   └── index.html
|
├── website2/
|   └── index.html
|
└── site1/
    └── public_html/
        └── index.html


/etc/letsencrypt/
└── live/
    └── devlinux.xyz/
        ├── fullchain.pem
        └── privkey.pem


/var/log/apache2/
├── devlinux_error.log
├── devlinux_access.log
```

```
├── website2_error.log
├── website2_access.log
├── site1_error.log
└── site1_access.log
```

One important correction: For multiple SSL websites, configure a separate `<VirtualHost *:443>` for each domain, with its own `ServerName` and SSL certificate. Also, ensure `Listen 443` is configured in `/etc/apache2/ports.conf`.

You can use this as your Ubuntu Apache practical guide for single website hosting, multiple website hosting, SSL configuration, and load balancing.